Virtualizing the LogUp* Pushforward (Preliminary Draft)

Hunter Lindauer

August, 2026

Abstract

In this work, we describe an approach to eliminate the commitment to transcript-derived LogUp* pushforward $Y = I_* \operatorname{eq}_r$. Rather than committing to the full pushforward, we take advantage of pushforward-pullback duality, representing Y virtually as a structured lookup. The resulting virtual evaluation claim is proven with a standard read-only-memory checking argument such as Shout. To support proofs of evaluation claims on both the LogUp* index map I and pushforward $I_* \operatorname{eq}_r$, we use a single committed representation of I, instantiated by commitments to the tensor components $I'_0, \ldots, I'_{d-1}$. This representation replaces the ordinary commitment to I and introduces no additional independent commitment to Y. The resulting scheme enables a smaller aggregate committed-polynomial footprint in relevant parameter regimes, and the d component commitments are independently computable and amenable to batching.

Contents

1 Introduction

In this work, we consider the commitment cost of the transcript-derived LogUp* pushforward $Y = I_* \operatorname{eq}_r$. In LogUp*, a proof of multilinear evaluation $\widetilde{Y}(\alpha)$ is made, requiring a commitment be made to Y

$$\widetilde{Y}(\alpha) = \langle Y, \operatorname{eq}_\alpha \rangle$$

We identify that for transcript-derived pushforward $Y = I_* \operatorname{eq}_r$, an explicit commitment is not required, and may instead be kept virtual by means of pushforward-pullback duality, $\langle I_* A, B \rangle = \langle A, I^* B \rangle$. The evaluation $\widetilde{Y}(\alpha)$ may be represented as a structured lookup on I,

$$\begin{aligned} \widetilde{Y}(\alpha) &= \langle I_* \operatorname{eq}_r, \operatorname{eq}_\alpha \rangle \\ &= \langle \operatorname{eq}_r, I^* \operatorname{eq}_\alpha \rangle \\ &= \sum_{i \in [n]} \operatorname{eq}_r[i] \cdot \operatorname{eq}_\alpha[I[i]] \end{aligned}$$

and proven via a standard read-only-memory / structured-lookup argument such as Shout, representing I as a matrix of one-hot vector rows (denoted I') which are subject to tensor decomposition. However, the duality stated up to now is not yet sufficient to eliminate the commitment on both Y since there is still a matter of proving evaluations of I, as well as I'.

The tensorized one-hot representation used to virtualize the transcript-dependent LogUp* pushforward can also serve as the committed representation of the witness index map itself, allowing both ordinary I-evaluation claims and virtual $I_* \operatorname{eq}_r$-evaluation claims to be discharged from the same committed representation. In this way, the commitments to both Y and I are replaced by a single committed representation of I, instantiated by commitments to the tensor components $I'_0, \ldots, I'_{d-1}$, which are not transcript-dependent:

$$\mathrm{Com}(I), \mathrm{Com}(I_* \operatorname{eq}_r) \longrightarrow \mathrm{Com}(I'_0), \ldots, \mathrm{Com}(I'_{d-1}).$$

The commitment size tradeoff is summarized quantitatively in the following table

	Ordinary LogUp*	Virtualized LogUp*
Table commitments	$1 \times T,\ \lvert T\rvert = m$	$1 \times T,\ \lvert T\rvert = m$
Index/pushforward commitments	$1 \times I,\ \lvert I\rvert = n$ $1 \times Y,\ \lvert Y\rvert = m$	$d \times I'_j,\ \lvert I'_j\rvert = nm^{1/d}$

Table 1: Comparison of Commitment Sizes

Totalling $2m + n$ field elements for standard LogUp*, and $m + dn \cdot m^{1/d}$ for our optimization. There is no transcript-dependent Y in our case, and the d component commitments are independently computable and amenable to batching. Increasing d reduces $m^{1/d}$ and the reconstruction depth, but increases the number of committed components and affects the Shout read-check cost; hence d should be chosen to balance these tradeoffs.

m	d	$\lvert I'_j\rvert$	$d\lvert I'_j\rvert$	Ordinary total	Virtualized total
2^{16}	2	262,144	524,288	132,096	589,824
2^{16}	4	16,384	65,536	132,096	131,072
2^{16}	8	4,096	32,768	132,096	98,304
2^{16}	16	2,048	32,768	132,096	98,304
2^{20}	2	1,048,576	2,097,152	2,098,176	3,145,728
2^{20}	4	32,768	131,072	2,098,176	1,179,648
2^{20}	5	16,384	81,920	2,098,176	1,130,496
2^{20}	10	4,096	40,960	2,098,176	1,089,536
2^{20}	20	2,048	40,960	2,098,176	1,089,536
2^{24}	2	4,194,304	8,388,608	33,555,456	25,165,824
2^{24}	3	262,144	786,432	33,555,456	17,563,648
2^{24}	4	65,536	262,144	33,555,456	17,039,360
2^{24}	6	16,384	98,304	33,555,456	16,875,520
2^{24}	8	8,192	65,536	33,555,456	16,842,752
2^{24}	12	4,096	49,152	33,555,456	16,826,368
2^{24}	24	2,048	49,152	33,555,456	16,826,368

Table 2: Total field elements committed to for $n = 2^{10}$ in standard and virtualized pushforward LogUp*.

1.1 Related Work

LogUp and LogUp*. LogUp [3] is a lookup argument that uses logarithmic-derivative identities to prove multiset membership relations. It may be optimized with GKR [4] to reduce the commitment cost of auxiliary polynomials when proving rational sum identities. LogUp* [7] is a variant of LogUp that applies the pushforward-pullback duality $\langle I_* A, B\rangle = \langle A, I^* B\rangle$ to prove indexed lookup relations. Unlike LogUp, which reduces claimed evaluations on the MLEs of indices to those on the MLEs of the table-reads, LogUp* reverses this order, reducing evaluation claims on the table-reads to those on the indices. In addition, LogUp* requires a proof of evaluation of the MLE of the pushforward vector $Y = I_* \operatorname{eq}_r$.

Unlike the table T and index map I, which are known to the prover in advance, the pushforward Y is defined in terms of a transcript-derived challenge r, and therefore must be committed to in the online portion of the protocol. This work aims to address this online commitment cost to Y.

TaSSLE TaSSLE [2] exploits tensor-decomposition of tables to reduce commitment cost in logarithmic-derivative lookup arguments. TaSSLE predates LogUp* and focuses on the original LogUp protocol. As a result, it does not discuss the online pushforward commitment cost in LogUp*. Moreover, TaSSLE targets the structure and commitment cost of the queried table, whereas this work targets the transcript-dependent LogUp* pushforward commitment. That said, the optimizations described in TaSSLE are compatible with ours.

MulPerm MulPerm [1] considers lookup maps via the relation R_ρ and its indicator $L_\rho(\mathbf{x}, \mathbf{y})$

$$R_\rho = \{(\mathbf{x}, \mathbf{y}) : \rho(\mathbf{x}) = \mathbf{y}\} \qquad L_\rho(\mathbf{x}, \mathbf{y}) = \begin{cases} 1 & (\mathbf{x}, \mathbf{y}) \in R_\rho \\ 0 & (\mathbf{x}, \mathbf{y}) \notin R_\rho \end{cases}$$

Membership in R_ρ is expressed by the lookup identity

$$\sum_{y \in \{0,1\}^\kappa} f(y)\, L_\rho(x, y) = g(x) \qquad \forall x \in \{0,1\}^\mu.$$

Then, with $\widetilde{\operatorname{eq}}_\alpha$ defined by verifier supplied challenge $\alpha \in \mathbb{F}^\mu$ yields the sumcheck identity (Equation 18)

$$\sum_{x \in \{0,1\}^\mu} \widetilde{\operatorname{eq}}_\alpha(x)\, g(x) = \sum_{x \in \{0,1\}^\mu} \sum_{y \in \{0,1\}^\kappa} \widetilde{\operatorname{eq}}_\alpha(x)\, f(y)\, \widetilde{L}_\rho(x, y).$$

Shout The Shout PIOP [5] is a memory-checking argument for proving that a read-only memory Val was accessed according to read-access pattern ra to produce read values rv. The MLE $\widetilde{ra}$ is that of a matrix whose rows are one-hot encodings of the indices at which elements of Val were accessed.

$$\widetilde{rv}(r') = \sum_{k \in \{0,1\}^{\log N},\, j \in \{0,1\}^{\log T}} \widetilde{\operatorname{eq}}(r', j) \cdot \widetilde{ra}(k, j) \cdot \widetilde{Val}(k)$$

Parameter d controls a tensor decomposition of the rows of ra, so that the prover commits to d smaller polynomials $\widetilde{ra}_i$ in place of a single $\widetilde{ra}$.

$$\widetilde{rv}(r') = \sum_{k=(k_1,\ldots,k_d) \in (\{0,1\}^{\log N/d})^d,\, j \in \{0,1\}^{\log T}} \widetilde{\operatorname{eq}}(r', j) \cdot \left(\prod_{i=0}^{d-1} \widetilde{ra}_i(k_i, j) \right) \cdot \widetilde{Val}(k)$$

When the read-access pattern is not committed in preprocessing, one-hotness of the ra rows (or ra_i if $d > 1$) must also be checked via sumchecks for Hamming weight one

$$1 = \sum_{k \in \{0,1\}^{\log K}} \widetilde{\mathrm{ra}}\left(k, r'_{\mathrm{cycle}}\right)$$

and booleanity

$$0 = \sum_{\substack{k \in \{0,1\}^{\log K} \\ j \in \{0,1\}^{\log T}}} \widetilde{\mathrm{eq}}(r,k)\, \widetilde{\mathrm{eq}}(r',j)\, \left(\widetilde{\mathrm{ra}}(k,j)^2 - \widetilde{\mathrm{ra}}(k,j)\right).$$

Shout supplies the tensorized one-hot read-check representation; our additional step is to reuse that representation to satisfy the separate I-evaluation claims required by LogUp*, thereby eliminating the independent commitments to both I and the transcript-derived pushforward.

1.2 Preliminaries

Multilinear Polynomials A multilinear polynomial is a multivariate polynomial that is linear in each variable. The multilinear equality polynomial is defined as

$$\mathrm{eq}(y_0, \ldots, y_{k-1}, x_0, \ldots, x_{k-1}) = \prod_{i=0}^{k-1} (y_i x_i + (1 - y_i)(1 - x_i))$$

And evaluates to 1 if $x = y$, otherwise 0. For a table $T \in \mathbb{F}^{2^k}$ and $x, y \in \{0,1\}^k$, its multilinear extension is

$$\widetilde{t}(x) = \sum_{y \in \{0,1\}^k} \widetilde{\mathrm{eq}}(y, x) \cdot T[y]$$

The evaluation of a multilinear extension $\widetilde{t}(x)$ at a point $\alpha \in \mathbb{F}^k$ is

$$\widetilde{t}(\alpha) = \sum_{y \in \{0,1\}^k} \widetilde{\mathrm{eq}}(\alpha, y) \cdot \widetilde{t}(y)$$

LogUp, LogUp* We recount the pushforward-pullback duality from LogUp* [7]. Let $n = 2^\nu$ and $m = 2^\mu$. Let A be a field-valued function on $[n] = \{0, \ldots, n-1\}$, and B on $[m] = \{0, \ldots, m-1\}$. The index map I is defined $I : [n] \to [m]$.

Define the pullback of B on I as

$$I^* B[i] = B[I[i]],$$

And the pushforward of A on I as

$$I_* A[j] = \sum_{i \,:\, I[i] = j} A[i]$$

The pullback and pushforward are dual (Lemma 1 [7]).

$$\langle I_* A, B \rangle = \langle A, I^* B \rangle$$

Consider the table T, index map I, pushforward Y of X, random challenge c. By Lemma 2 [7], for committed Y, random challenge c, the equality below implies the claim $Y = I_* X$ with soundness error $\frac{n+m}{|\mathbb{F}|}$

$$\sum_{0 \le i < n} \frac{X[i]}{c - I[i]} = \sum_{0 \le j < m} \frac{Y[j]}{c - j}$$

Translating to the setting of the LogUp-GKR protocol [4], let $X = \mathrm{eq}_r$. The prover commits to I, T, $I_* \mathrm{eq}_r$, then claims the pullback evaluation $\widetilde{I^*T}(r) = e$ for $r \in \mathbb{F}^\nu$. The validity of the pushforward is proven by invoking GKR to enforce the rational sum identity below, reducing to evaluation claims $\widetilde{I}(\beta)$, $\widetilde{Y}(\alpha)$, $\widetilde{\mathrm{eq}}_r(\beta)$ for $\beta \in \mathbb{F}^\nu$ and $\alpha \in \mathbb{F}^\mu$.

$$\sum_{0 \le i < n} \frac{\mathrm{eq}_r[i]}{c - I[i]} = \sum_{0 \le j < m} \frac{I_* \mathrm{eq}_r[j]}{c - j}$$

The eval claim e is proven with a sumcheck of the form $\langle T, I_* \mathrm{eq}_r \rangle - e = 0$, reducing to evaluation claims on $\widetilde{Y}(\alpha')$, $\widetilde{T}(\alpha')$ for $\alpha' \in \mathbb{F}^\mu$. The prover opens commitments with a batched proof of evaluation claims $\widetilde{I}(\beta)$, $\widetilde{Y}(\alpha)$, $\widetilde{Y}(\alpha')$, $\widetilde{T}(\alpha')$. Doing so proves the evaluation claim on the pullback I^*T while avoiding commitment to the full pullback, by taking advantage of the pullback-pushforward duality.

2 Reducing the Pushforward Commitment Cost

Let $n = 2^\nu$, $m = 2^\mu$, with $d \mid \mu$. Write $[n] = \{0, \ldots, n-1\}$, $[m] = \{0, \ldots, m-1\}$ over the integers, and $x \in \{0,1\}^\nu$, $z \in \{0,1\}^\mu$ for their respective canonical coordinates on the Boolean hypercube.

Let $I \in \mathbb{F}^n$ be the index map, and $T \in \mathbb{F}^m$ be the table. Consider the LogUp* pushforward $Y = I_* \mathrm{eq}_r$. Observe that Y is defined in terms of a Fiat-Shamir challenge $r \in \mathbb{F}^\nu$. Since r is itself transcript-derived, the commitment to Y cannot be made before r is sampled. If the table T is large, this commitment may become a prohibitively expensive online commitment cost for the prover.

Lemma 2.1. *Let $n = 2^\nu$, $m = 2^\mu$, table $T \in \mathbb{F}^m$, indices $I \in \mathbb{F}^n$, and pushforward $Y = I_* \mathrm{eq}_r$ for transcript-derived challenge $r \in \mathbb{F}^\nu$. The evaluation $\widetilde{Y}(\alpha)$ for $\alpha \in \mathbb{F}^\mu$ is equivalently represented as the structured lookup $\widetilde{Y}(\alpha) = \sum_{i \in [n]} \mathrm{eq}_r[i] \cdot \mathrm{eq}_\alpha[I[i]]$*

Proof. Notice that the evaluation $\widetilde{Y}(\alpha)$ may be computed as an inner product that is subject to the same pushforward-pullback duality as before

$$\begin{aligned}
\widetilde{Y}(\alpha) &= \langle Y, \mathrm{eq}_\alpha \rangle \\
&= \langle I_* \mathrm{eq}_r, \mathrm{eq}_\alpha \rangle \\
&= \langle \mathrm{eq}_r, I^* \mathrm{eq}_\alpha \rangle \\
&= \sum_{i \in [n]} \mathrm{eq}_r[i] \cdot \mathrm{eq}_\alpha[I[i]]
\end{aligned}$$

□

Lemma 2.1 establishes that $\widetilde{Y}(\alpha)$ may be viewed as an evaluation of a structured lookup into read-only memory eq_α by indices $I[i]$. This enables us to use memory checking arguments such as Shout to prove their validity. Let the matrix $I' \in \mathbb{F}^{n \times m}$ contain rows which are the one-hot encodings of the entries $I[i]$. Construct the following instantiation the Shout [5] read-check argument with $d = 1$

$$\widetilde{Y}(\alpha) = \sum_{x\in\{0,1\}^\nu,\, z\in\{0,1\}^\mu} \widetilde{\mathrm{eq}}_r(x)\cdot\widetilde{\mathrm{eq}}_\alpha(z)\cdot\widetilde{I'}(z,x)$$

And adjusting to general-d

$$\widetilde{Y}(\alpha) = \sum_{x\in\{0,1\}^\nu, z=(z_0,\ldots,z_{d-1})\in(\{0,1\}^{\mu/d})^d} \widetilde{\mathrm{eq}}_r(x)\cdot\widetilde{\mathrm{eq}}_\alpha(z)\cdot\prod_{j=0}^{d-1}\widetilde{I'_j}(z_j,x)$$

Described up to now is the Shout read check for the identity in Lemma 2.1. In the full Shout PIOP protocol, the polynomials $\widetilde{I'_j}$ require commitments and to be checked for one-hotness using the usual Shout sumcheck arguments for booleanity and Hamming weight one. Since the standard LogUp* protocol includes a commitment to I, we demonstrate that this commitment may be replaced for just that of each $\widetilde{I'_j}$.

Lemma 2.2. *The d-dimensional one-hot matrix $(I'_0,\ldots,I'_{d-1}) \in \{0,1\}^{\mu/d}\times\{0,1\}^\nu$ determines arbitrary evaluations of $\widetilde{I}(\beta)$ for $\beta\in\mathbb{F}^\nu$ by*

$$\widetilde{I}(\beta) = \sum_{j=0}^{d-1}\sum_{t=0}^{\mu/d-1} 2^{j\mu/d+t}\sum_{z_j\in\{0,1\}^{\mu/d}} z_{j,t}\,\widetilde{I'_j}(z_j,\beta).$$

Proof. By construction, the d-dimensional one-hot matrices $(I'_0,\ldots,I'_{d-1})$ are related to the one-dimensional one-hot matrix I' by

$$I'(z,x) = \prod_{j=0}^{d-1} I'_j(z_j,x), \qquad z=(z_0,\ldots,z_{d-1})\in(\{0,1\}^{\mu/d})^d, \quad x\in\{0,1\}^\nu$$

Since for all $x\in\{0,1\}^\nu$, $I'(\cdot,x)$ is the one-hot encoding of $I[x]\in[m]$, we may view the reconstruction of $I[x]$ from $I'(z,x)$ as a lookup into $[m]$ via the one-hot row $I'(\cdot,x)$. In this way, an evaluation of I using I' may be done via

$$\widetilde{I}(\beta) = \sum_{x\in\{0,1\}^\nu,\, z\in\{0,1\}^\mu} \widetilde{\mathrm{eq}}(\beta,x)\cdot I'(z,x)\cdot z.$$

Since $I'(z,x)$ evaluates to 1 only when z encodes the bit decomposition of $I[x]$, we may replace the scalar z by its bit string recombination

$$z = \sum_{j=0}^{d-1}\sum_{t=0}^{\mu/d-1} 2^{j\mu/d+t} z_{j,t},$$

yielding

$$I[x] = \sum_{z\in\{0,1\}^\mu} I'(z,x)\left(\sum_{j=0}^{d-1}\sum_{t=0}^{\mu/d-1} 2^{j\mu/d+t} z_{j,t}\right).$$

Substituting the tensor-product recombincation of each I'_j into I',

$$I[x] = \sum_{z\in\{0,1\}^\mu}\left(\prod_{j=0}^{d-1} I'_j(z_j,x)\right)\left(\sum_{j=0}^{d-1}\sum_{t=0}^{\mu/d-1} 2^{j\mu/d+t} z_{j,t}\right)$$

Distributing and rerarrange the sums,

$$I[x] = \sum_{j=0}^{d-1} \sum_{t=0}^{\mu/d-1} 2^{j\mu/d+t} \sum_{z\in\{0,1\}^{\mu}} z_{j,t} \prod_{k=0}^{d-1} I'_k(z_k, x).$$

The inner sum factors over z as

$$\sum_{z_j\in\{0,1\}^{\mu/d}} z_{j,t} I'_j(z_j, x) \cdot \prod_{k\neq j} \Big(\sum_{z_k\in\{0,1\}^{\mu/d}} I'_k(z_k, x) \Big),$$

and one-hotness $\sum_{z_j\in\{0,1\}^{\mu/d}} I'_j(z_j, x) = 1$ collapses every factor except the j-th, yielding

$$I[x] = \sum_{j=0}^{d-1} \sum_{t=0}^{\mu/d-1} 2^{j\mu/d+t} \sum_{z_j\in\{0,1\}^{\mu/d}} z_{j,t}\, I'_j(z_j, x)$$

Taking the multilinear extension in x and evaluating at $\beta \in \mathbb{F}^{\nu}$ gives the claim. □

Theorem 2.3. *Given commitments to $(I'_0, \ldots, I'_{d-1})$, a claim $v = \widetilde{I}(\beta)$ for $\beta \in \mathbb{F}^{\nu}$ can be reduced by a (μ/d)-round sumcheck to evaluation claims on the committed I'_j at a common random point $\rho \in \mathbb{F}^{\mu/d}$.*

Proof. Starting from Lemma 2.2 and reordering the sums,

$$\widetilde{I}(\beta) = \sum_{z\in\{0,1\}^{\mu/d}} \left(\sum_{t=0}^{\mu/d-1} 2^t z_t \right) \left(\sum_{j=0}^{d-1} 2^{j\mu/d}\, \widetilde{I}'_j(z, \beta) \right).$$

Run sumcheck over z. If it terminates at $\rho \in \mathbb{F}^{\mu/d}$, the terminal claim is

$$\left(\sum_{t=0}^{\mu/d-1} 2^t \rho_t \right) \left(\sum_{j=0}^{d-1} 2^{j\mu/d}\, \widetilde{I}'_j(\rho, \beta) \right),$$

which reduces the original claim to $\widetilde{I}'_j(\rho, \beta)$ for $j \in [d]$, all at the same point and therefore batch-openable with the underlying PCS. The summed polynomial has individual degree at most 2, giving sumcheck soundness error at most $\frac{2\mu/d}{|\mathbb{F}|}$. □

Theorem 2.4. *Let $Y = I_* \operatorname{eq}_r$, and let $(I'_0, \ldots, I'_{d-1})$ be a valid committed d-dimensional one-hot representation of I. Then the commitments to the I'_j are sufficient to replace both the commitments on I and Y.*

Proof. By Lemma 2.1 and the Shout read-check construction immediately following it, every terminal LogUp* evaluation claim

$$\widetilde{Y}(\alpha) = \widetilde{I_* \operatorname{eq}_r}(\alpha)$$

can be proven directly from the committed I'_j, so no independent commitment to Y is required. By Lemma 2.2, arbitrary evaluations $\widetilde{I}(\beta)$ are determined by the tensorized representation. By Theorem 2.3, each such claim $\widetilde{I}(\beta)$ can be reduced by reconstruction sumcheck to evaluation claims $\widetilde{I}'_j(\rho, \beta)$ on the already committed I'_j, so no independent commitment to I is required. Thus

$$\operatorname{Com}(I'_0), \ldots, \operatorname{Com}(I'_{d-1}) \Rightarrow \text{all required } I\text{-evaluation claims}$$

and

$$\mathrm{Com}(I'_0), \ldots, \mathrm{Com}(I'_{d-1}) \Rightarrow \text{all required virtual } Y\text{-evaluation claims.}$$

The ordinary LogUp* commitment to I and the transcript-dependent commitment to $Y = I_* \,\mathrm{eq}_r$ may both be omitted and replaced by the commitments to the tensorized one-hot representation. □

Protocol: Let $n = 2^\nu$, $m = 2^\mu$ with $d \mid \mu$. Write $Y := I_* \,\mathrm{eq}_r$, and let $I'_j \in \mathbb{F}^{n \times m^{1/d}}$ for $j \in \{0, \ldots, d-1\}$ be the d-dimensional one-hot decomposition of the witness index map I. The table T may be committed in preprocessing. The commitments $\mathrm{Com}(I'_0), \ldots, \mathrm{Com}(I'_{d-1})$ are fixed before r.

- In the witness-commitment phase, before the transcript-derived challenge r, P commits to $I'_0, \ldots, I'_{d-1}$ as the canonical committed representation of I. There is no commitment to Y. P and V also run the standard Shout / Twist-and-Shout booleanity and Hamming-weight-one checks on each I'_j [5, Figure 8], reducing to evaluation claims on the I'_j that are deferred to the final batched opening.
- After r is sampled, P runs ordinary LogUp* only until its terminal evaluation claims are obtained, treating $Y = I_* \,\mathrm{eq}_r$ as virtual throughout. This yields the usual final evaluation claims on Y and T, together with any evaluation claims on I.
- For every terminal claim $v_Y = \widetilde{Y}(\alpha)$ with $\alpha \in \mathbb{F}^\mu$, P and V discharge it with the Shout read-check

$$v_Y = \sum_{\substack{x \in \{0,1\}^\nu \\ z = (z_0, \ldots, z_{d-1}) \in (\{0,1\}^{\mu/d})^d}} \widetilde{\mathrm{eq}}_r(x)\, \widetilde{\mathrm{eq}}_\alpha(z) \prod_{j=0}^{d-1} \widetilde{I}'_j(z_j, x),$$

  reducing to evaluation claims $v_{I,j} := \widetilde{I}'_j(\rho_j, \rho_x)$ for $j \in \{0, \ldots, d-1\}$, where $\rho_x \in \mathbb{F}^\nu$ and each $\rho_j \in \mathbb{F}^{\mu/d}$. Writing $\rho_z = (\rho_0, \ldots, \rho_{d-1}) \in (\mathbb{F}^{\mu/d})^d$, V checks the final sumcheck claim is consistent with $\widetilde{\mathrm{eq}}_r(\rho_x) \cdot \widetilde{\mathrm{eq}}_\alpha(\rho_z) \cdot \prod_{j=0}^{d-1} v_{I,j}$, where $\widetilde{\mathrm{eq}}_r(\rho_x)$ and $\widetilde{\mathrm{eq}}_\alpha(\rho_z)$ are verifier-computed.
- For every terminal LogUp* claim $v_I = \widetilde{I}(\beta)$ with $\beta \in \mathbb{F}^\nu$, P and V discharge it with the reconstruction sumcheck of Theorem 2.3. The sumcheck terminates at a common $\zeta \in \mathbb{F}^{\mu/d}$, reducing to evaluation claims $v^{\mathrm{rec}}_{I,j} := \widetilde{I}'_j(\zeta, \beta)$ for all $j \in \{0, \ldots, d-1\}$.
- P batch-opens all resulting evaluation claims on the already committed I'_j together with the remaining LogUp* claims on T. V checks the openings are consistent with the claimed evaluations.

Figure 1: Virtualizing LogUp*'s terminal $I_* \,\mathrm{eq}_r$ evaluations.

Theorem 2.5. *The protocol of Figure 1 is perfectly complete. Let q_Y be the number of independently invoked Shout read-checks for terminal Y-evaluations, and q_I the number of reconstruction sumchecks for terminal I-evaluations. Identifying Shout's memory size K with $m = 2^\mu$ and its*

number of reads T with $n = 2^\nu$, the soundness error is at most

$$\epsilon_{\text{VirtualLogUp}*} \le \epsilon_{\text{id}} + \epsilon_{\text{LogUpGKR}} + q_Y \epsilon_{\text{Shout}} + \epsilon_{\text{1Hot}} + q_I \frac{2\mu/d}{|\mathbb{F}|} + \epsilon_{\text{PCS}},$$

where $\epsilon_{\text{id}} = \frac{n+m}{|\mathbb{F}|}$ is the soundness error of the LogUp pushforward rational-identity check (Lemma 2 of [7]), $\epsilon_{\text{LogUpGKR}}$ is the residual soundness of the remaining LogUp-GKR and $\langle T, Y\rangle$-sumcheck components (left symbolic), $\epsilon_{\text{Shout}} = \frac{(d+2)\nu+2\mu}{|\mathbb{F}|}$ is the core read-check error of Figure 7 (Theorem 3 of [5]), $\epsilon_{\text{1Hot}} = \frac{4d\nu+6\mu}{|\mathbb{F}|}$ is the one-hot validity error of Figure 8 (Theorem 3 of [5]), counted once because the I'_j are committed and checked a single time before r, and ϵ_{PCS} is the soundness error of the underlying batched PCS openings, counted once. For the special case $d = 1$, Theorem 2 of [5] already packages the $d = 1$ read-check together with one-hot validity at error $\frac{6\mu+4\nu}{|\mathbb{F}|}$; that combined bound must not be added on top of ϵ_{1Hot}.*

Proof. Acceptance of the one-hot checks implies that $(I'_0, \dots, I'_{d-1})$ define a unique index map I. Acceptance of the reconstruction sumchecks of Theorem 2.3 then implies that every claimed $\widetilde{I}(\beta)$ is an evaluation of that I. Acceptance of the Shout read-checks, using Lemma 2.1, implies that every claimed

$$\widetilde{Y}(\alpha) = \widetilde{I_* \operatorname{eq}_r}(\alpha)$$

is the corresponding virtual pushforward evaluation of the same I. Hence the remaining transcript is equivalent to a valid LogUp* transcript for the induced I and $Y = I_* \operatorname{eq}_r$, except that I and Y are represented virtually through the committed I'_j (Theorem 2.4). A union bound over the isolated identity error ϵ_{id}, the residual LogUp-GKR error, the q_Y core Shout read-checks, the single one-hot check, the q_I degree-at-most-2 reconstruction sumchecks of Theorem 2.3, and the PCS openings yields the stated bound. □

2.1 Cost comparison

The index map I is part of the witness and is not preprocessing-known. The shared commitment to $T \in \mathbb{F}^m$ may be preprocessing-known and is kept outside the main comparison below. (Y is transcript-derived and therefore cannot be committed before r, which motivates the construction, but timing is not the cost metric of this subsection.)

Theorem 2.6. *Let $n = 2^\nu$, $m = 2^\mu$, and $d \mid \mu$. Ignoring the shared commitment to T, ordinary LogUp* commits to $\text{Com}(I)$ with $I \in \mathbb{F}^n$ and $\text{Com}(Y)$ with $Y = I_* \operatorname{eq}_r \in \mathbb{F}^m$. The virtualized construction of Figure 1 replaces both with $\text{Com}(I'_0), \dots, \text{Com}(I'_{d-1})$, $I'_j \in \mathbb{F}^{n \times m^{1/d}}$. This is the commitment-profile transformation*

$$\boxed{1 \text{ polynomial of size } n+1 \text{ polynomial of size } m \longrightarrow d \text{ polynomials, each of size } nm^{1/d}.}$$

Let $C_{\text{com}}(N)$ be the cost of committing to a size-N polynomial and $C_{\text{open}}(N, q)$ the cost of a q-point batched opening of a size-N polynomial. The affected commitment and opening costs transform from $C_{\text{com}}(n) + C_{\text{com}}(m) + C_{\text{open}}(n, q_I) + C_{\text{open}}(m, q_Y)$ to $d\, C_{\text{com}}(nm^{1/d}) + C_{\text{open}}(nm^{1/d}, q')$, together with q_I reconstruction sumchecks of μ/d rounds and individual degree at most 2, each terminating in d batchable evaluations of the I'_j, and q_Y Shout read-checks, each terminating in evaluations of the committed I'_j.

Proof. Immediate from Theorem 2.4, Theorem 2.3, and the Shout read-check of Lemma 2.1. □

The tensorized representation has smaller aggregate polynomial length than the two ordinary polynomials if and only if $dnm^{1/d} < n+m$. In the regime $m \gg n$ this is approximately $n < \frac{m^{1-1/d}}{d}$. This condition is not necessary for the construction to be useful: even when it does not reduce aggregate polynomial length, eliminating the size-m pushforward in favor of a structured read-check may still be advantageous. The parameter d trades $|I'_j| = nm^{1/d}$, the number of committed polynomials, reconstruction depth μ/d, and Shout cost. We do not claim a universally optimal d. Representative sizes for $n = 2^{10}$ appear in Table 2.

	Ordinary LogUp*	Virtualized LogUp*
Table commitments	$1 \times T,\ \|T\| = m$	$1 \times T,\ \|T\| = m$
Index/pushforward commitments	$1 \times I,\ \|I\| = n$ $1 \times Y,\ \|Y\| = m$	$d \times I'_j$ $\|I'_j\| = nm^{1/d}$

Table 3: Polynomial commitments. The shared commitment to T is displayed for completeness and is excluded from Theorem 2.6.

The corresponding proof-side tradeoff, separate from the commitment profile above, is as follows.

	Ordinary LogUp*	Virtualized LogUp*
$\widetilde{T}(\gamma)$	PCS opening of committed T	PCS opening of committed T
$\widetilde{I}(\beta)$	direct PCS opening	(μ/d)-round reconstruction sumcheck, then batched openings of the I'_j
$\widetilde{Y}(\alpha)$	PCS opening of committed Y	Shout read-check, then batched openings of the I'_j

Table 4: Proof-side cost of discharging terminal evaluation claims.

The virtualized Y-evaluation inherits the prover work of the Shout read-check, which we leave qualitative here.

References

[1] Benedikt Bünz, Jessica Chen, Zachary DeStefano, and Binyi Chen. Almost linear-time permutation check. Cryptology ePrint Archive, Paper 2025/1850, 2025. URL: https://eprint.iacr.org/2025/1850.

[2] Tesseract Dore. TaSSLE: Lasso for the commitment-phobic. Cryptology ePrint Archive, Paper 2024/1075, 2024. URL: https://eprint.iacr.org/2024/1075.

[3] Ulrich Haböck. Multivariate lookups based on logarithmic derivatives. Cryptology ePrint Archive, Paper 2022/1530, 2022. URL: https://eprint.iacr.org/2022/1530.

[4] Shahar Papini and Ulrich Haböck. Improving logarithmic derivative lookups using GKR. Cryptology ePrint Archive, Paper 2023/1284, 2023. URL: https://eprint.iacr.org/2023/1284.

[5] Srinath Setty and Justin Thaler. Twist and shout: Faster memory checking arguments via one-hot addressing and increments. Cryptology ePrint Archive, Paper 2025/105, 2025. URL: https://eprint.iacr.org/2025/105.

[6] Srinath Setty, Justin Thaler, and Riad Wahby. Unlocking the lookup singularity with lasso. Cryptology ePrint Archive, Paper 2023/1216, 2023. URL: https://eprint.iacr.org/2023/1216.

[7] Lev Soukhanov. Logup*: faster, cheaper logup argument for small-table indexed lookups. Cryptology ePrint Archive, Paper 2025/946, 2025. URL: https://eprint.iacr.org/2025/946.